DS105 FINAL PROJECT DOCUMENTATION

Time-Based Browsing Pattern Analyzer

using Deep Learning with RAM Usage Correlation

Author	Sathishkumar CB
Project Type	DS105 Final Project
Domain	Behavior Analytics · Digital Wellbeing · Productivity Analytics
Tech Stack	Python · Pandas · NumPy · SQLite · Scikit-learn · TensorFlow · Keras · psutil · Streamlit
Data Source	Self-generated — Local Browser SQLite DB + psutil RAM Logger
DL Models	Autoencoder (Anomaly Detection) · LSTM/GRU (Next-Category Prediction)
Deliverables	Source Code · Processed Datasets · Final Report · README · Streamlit Dashboard

"Understand how you browse. Optimize how you perform."

TABLE OF CONTENTS

1. **Introduction** 3
2. **Problem Statement** 3
3. **Objectives** 3
4. **Scope of the Project** 4
5. **System Architecture** 4
6. **Methodology** 5
   - 6.1 Data Collection 5
   - 6.2 Data Preprocessing 5
   - 6.3 Sessionization 5
   - 6.4 Feature Engineering 6
   - 6.5 Clustering (Machine Learning) 6
   - 6.6 Deep Learning 6
   - 6.7 RAM Correlation 7
   - 6.8 Recommendation Engine 7
7. **Tools and Technologies** 7
8. **Dataset Description** 8
9. **Results and Observations** 8
10. **Output** 9
11. **Advantages** 9
12. **Limitations** 9
13. **Future Enhancements** 10
14. **Conclusion** 10
15. **References** 10
16. **Author** 10

1. Introduction

In today's digital world, users spend significant time browsing the internet — often without understanding their usage patterns or the impact on system performance. Excessive or unstructured browsing can lead to reduced productivity, increased cognitive overload, and high memory consumption.

This project builds a comprehensive AI-based system that analyzes browsing behavior over a configurable time window (last 3, 4, or 5 days) and correlates it with real-time system RAM metrics. By combining data engineering, machine learning, and deep learning, the system identifies behavioral patterns, clusters user sessions, detects anomalous activity, and delivers actionable personalized recommendations.

The project is grounded in three core pillars:

- Behavioral Intelligence — understanding how, when, and what users browse.
- System Performance Intelligence — correlating browsing with memory usage.
- Prescriptive Analytics — turning data insights into practical recommendations.

2. Problem Statement

Users currently lack visibility into:

- How browsing habits vary across hours, days, and sessions.
- Which websites or categories consume the most system RAM.
- When they are most productive versus most distracted.
- What constitutes abnormal or anomalous browsing behavior.

The goal is to build an AI-based system that:

1. Analyzes browsing patterns — time-of-day, day-of-week, and session-level behaviour.
2. Identifies dominant website categories (social, video, learning, productivity, etc.).
3. Clusters user sessions into interpretable behaviour groups.
4. Detects anomalies using a deep learning Autoencoder or LSTM model.
5. Correlates browsing behaviour with RAM usage via nearest-timestamp join.
6. Provides at least 5 traceable, metric-backed actionable recommendations.

3. Objectives

- Extract and preprocess browser history data from Chrome/Edge SQLite databases.
- Create browsing sessions based on configurable inactivity time gaps.
- Engineer rich session-level features (ratios, switching rate, RAM stats).
- Apply unsupervised clustering (KMeans / GMM / DBSCAN) to discover behaviour groups.
- Detect anomalous sessions using an Autoencoder or predict next-category using LSTM.
- Monitor and correlate system + browser RAM usage with browsing categories.
- Generate meaningful, explainable, and traceable recommendations.
- Deliver a final report and an interactive Streamlit dashboard.

4. Scope of the Project

This project focuses on:

- Personal browsing data analysis (single-user, local machine).
- Time-based behavior analytics over a selectable 3/4/5-day window.
- RAM usage monitoring correlated at the session and category level.
- Machine learning (clustering) and deep learning (LSTM / Autoencoder) implementation.
- Privacy-preserving data handling — only domain-level data is retained.

Limitations:

- Works on local machine data only — no cloud or cross-device tracking.
- Browsing history records visits, not exact dwell time per page.
- Accuracy depends on browser history availability and RAM log coverage.
- Does not cover mobile or incognito browsing sessions.

5. System Architecture

The system consists of eight interconnected layers, each responsible for a distinct stage of the pipeline:

Layer 1	**Data Collection**	Extract Chrome/Edge SQLite history + log system RAM via psutil every 5–10 seconds.
Layer 2	**Data Preprocessing**	URL sanitization, domain extraction, category mapping (180+ domains), timezone conversion, derived columns.
Layer 3	**Sessionization**	Split events into sessions on 15-minute inactivity gaps; compute per-session feature vectors.
Layer 4	**Feature Engineering**	Compute category ratios, switching rate, hour_start, is_weekend, RAM means and peaks.
Layer 5	**Machine Learning**	KMeans / GMM / DBSCAN clustering with Silhouette Score evaluation and ARI stability check.
Layer 6	**Deep Learning**	Autoencoder for anomaly detection OR LSTM/GRU for next-category sequence prediction.
Layer 7	**Analytics & Recommendations**	RAM correlation analysis, rule + model-signal recommendation engine, ≥ 5 traceable tips.
Layer 8	**Visualization Dashboard**	Streamlit app with 5 interactive pages: Overview, Hourly, Clusters, RAM, Anomalies, Recs.

Data Flow: Raw SQLite DB → Preprocessed CSV → Sessions CSV → Clustered CSV → DL Enriched CSV → Final Report + Dashboard

6. Methodology

The methodology follows a structured 7-module pipeline. Each module is independently runnable and produces an output that feeds the next stage.

6.1 Data Collection

6.1.1 Browsing History Extraction

Chrome and Microsoft Edge store browsing history in an SQLite database file named 'History'. Since the browser keeps this file locked during use, the system copies it to a temporary file before querying.

- Source file: ~/.config/google-chrome/Default/History (Linux) or equivalent on Windows.
- Chrome timestamps are stored as microseconds since 1601-01-01 UTC — converted to local datetime.
- Each row represents a URL visit with timestamp, URL, and page title.
- Invalid URLs (non-HTTP), duplicates, and blank domains are removed.
- Extraction completeness is logged: % of records with a valid domain.

6.1.2 RAM Usage Logging

A background psutil-based logger samples system metrics at a configurable interval (default: every 8 seconds).

- system RAM used (MB) and available (MB).
- Browser process RAM — sum of RSS memory of all Chrome/Edge child processes.
- CPU utilization (%) — optional metric for performance correlation.
- Each row is timestamped in ISO-8601 UTC format and appended to ram_log.csv.

6.2 Data Preprocessing

The preprocessing pipeline ensures data quality, privacy compliance, and feature readiness.

- Remove duplicate rows and rows with missing or invalid URLs.
- Extract bare domain from URL (strip www., query strings, path fragments).
- Map domain → category using a curated table of 180+ entries (social, video, learning, productivity, shopping, news, search, gaming, finance, reference, tools, other).
- Convert all timestamps to the user's local timezone (configurable in config.yaml).
- Add derived columns: hour (0–23), date, day_name, is_weekend, time_block (early_morning / morning / lunch / afternoon / evening / late_night).
- Filter records to the selected time window (last 3, 4, or 5 days).

Privacy: Only domain names and category labels are retained. Full URLs with paths and query parameters are stripped before any CSV is saved.

6.3 Sessionization

Browsing events are grouped into sessions using an inactivity threshold rule:

Rule: If the time gap between two consecutive events exceeds 15 minutes → a new session begins. (Threshold is configurable in config.yaml.)

For each session, the following summary statistics are computed:

- n_visits — total number of URL visits.

- duration_min — session length in minutes.
- n_unique_domains and n_unique_categories.
- dominant_category — most frequently visited category.
- Category ratios — social_ratio, video_ratio, learning_ratio, productivity_ratio, shopping_ratio, other_ratio.
- switching_rate — fraction of consecutive category transitions that differ (0 = monotonic, 1 = fully alternating).
- hour_start, is_weekend, time_block.
- categories_sequence — JSON list of categories visited (input for LSTM model).

A manual spot-check of the first 5 sessions is logged to the console for verification.

6.4 Feature Engineering

In addition to session-level features computed during sessionization, RAM metrics are joined using pd.merge_asof (nearest-timestamp merge with a configurable tolerance of ±60 seconds):

- browser_ram_mean — mean browser RAM (MB) during the session.
- browser_ram_peak — maximum browser RAM (MB) during the session.
- ram_used_mean and ram_used_peak — system-level RAM equivalents.

The complete feature set (13–17 features per session depending on RAM availability) is used for both clustering and deep learning.

6.5 Clustering (Machine Learning)

All session features are standardized with StandardScaler before clustering. Three algorithms are supported (selectable in config.yaml):

Algorithm	Best For	Evaluation
KMeans	Well-separated, roughly equal-size clusters. Most interpretable.	Silhouette Score + ARI stability.
GMM	Soft/overlapping clusters, ellipsoidal shapes.	Log-likelihood + Silhouette Score.
DBSCAN	Arbitrary shapes, automatic outlier detection.	Number of clusters + noise ratio.

Each cluster is automatically labeled using interpretation rules: dominant_category + avg hour + session intensity (light / moderate / heavy). Example labels: 'heavy evening social session', 'light morning productivity session'.

6.6 Deep Learning

Two deep learning options are provided; at least one must be implemented:

Option A — LSTM / GRU Next-Category Prediction

The LSTM model treats each session as a sequence of category visits and learns to predict the next category.

- Input: sliding window of 5 consecutive category labels (integer-encoded).
- Architecture: Embedding(vocab) → LSTM(64 units) → Dense(softmax).
- Training: 80/20 train-test split, Adam optimizer, sparse categorical cross-entropy.
- Metrics: accuracy, macro F1-score, confusion matrix, baseline comparison (most-common-class).

Option B — Autoencoder Anomaly Detection

The Autoencoder learns a compressed representation of 'normal' session behaviour. Sessions that are hard to reconstruct are flagged as anomalies.

- Input: standardized session feature vector (13–17 dimensions).
- Architecture: Dense(enc_dim×2, ReLU) → Dense(enc_dim, ReLU) → Dense(enc_dim×2, ReLU) → Dense(input_dim, linear).
- Training: MSE loss, Adam optimizer, 10% validation split.
- Anomaly score: mean squared reconstruction error per session.
- Top-N anomalies flagged with human-readable reasons: late-night browsing, high tab-switching, RAM spike, social media heavy, unusually high visit count.

6.7 RAM Correlation

Browsing events are merged with RAM logs using pd.merge_asof (direction='nearest', tolerance=60s). This produces two analysis views:

- Category-wise RAM stats — mean and peak browser RAM per category (saved to reports/category_ram_stats.csv).
- Session-wise RAM stats — mean and peak RAM for each session (merged into sessions CSV).
- Top 3 memory-heavy categories are identified and logged for the recommendation engine.

6.8 Recommendation Engine

The recommendation engine combines rule-based logic with model signals to generate ≥ 5 personalized, traceable recommendations. Every recommendation includes a metric-backed evidence statement.

Recommendation	Trigger Condition	Priority
Reduce late-night social media	social_ratio > 0.45 after 10 PM	High
Schedule focus blocks	Peak distraction hour identified	High
Manage RAM from heavy sites	browser_ram_peak > 1500 MB	Medium
Increase learning time	distraction_ratio > 2× learning_ratio	Medium
Reduce context switching	switching_rate > 0.70 in ≥ 1 session	Medium
Review anomalous sessions	Autoencoder flags ≥ 1 anomaly session	Low
Protect natural learning hour	Hour with highest learning	Low

	site visits	

7. Tools and Technologies

Category	Tool / Library	Purpose
Programming	Python 3.10+	Core language for all modules
Data Processing	Pandas, NumPy	DataFrames, array operations, merges
Database	SQLite3, shutil	Browser history extraction (copy + query)
Preprocessing	pytz, urllib.parse	Timezone conversion, URL parsing
Machine Learning	Scikit-learn	KMeans, GMM, DBSCAN, StandardScaler, metrics
Deep Learning	TensorFlow / Keras	LSTM, GRU, Autoencoder model building
System Monitor	psutil	System + browser RAM and CPU sampling
Visualization	Matplotlib, Seaborn	Charts for reports and notebooks
Dashboard	Streamlit	Interactive 5-page web dashboard
Configuration	PyYAML	Central config.yaml management
Packaging	Git, requirements.txt	Version control and environment reproducibility

8. Dataset Description

All data is self-generated from the learner's local machine. Three CSV files form the core dataset:

8.1 browsing_history.csv

Column	Type	Description
timestamp	datetime	Visit time (UTC, converted to local TZ)
url	string	Sanitized URL (query strings removed)
domain	string	Extracted bare domain (e.g. youtube.com)
category	string	Mapped category (social/video/learning/...)
browser	string	Source browser (chrome / edge)

hour	int	Hour of visit (0–23)
date	date	Calendar date of visit
day_name	string	Day of week (Monday … Sunday)
time_block	string	Named time block (morning / evening / ...)

8.2 ram_log.csv

Column	Type	Description
timestamp	datetime	Sample time (UTC ISO-8601)
ram_used_mb	float	System RAM currently in use (MB)
ram_available_mb	float	System RAM available (MB)
browser_ram_mb	float	Total browser process RSS (MB)
cpu_percent	float	CPU utilization % (optional)

8.3 domain_category_map.csv

A curated mapping of 180+ domains to categories. Learners can extend this file to add custom mappings. Categories include: social, video, learning, productivity, shopping, news, search, gaming, finance, reference, tools, other.

9. Results and Observations

Upon running the complete pipeline, the system produces the following insights:

Insight Type	What Is Produced
Top Domains	Ranked list of most visited websites with visit counts and categories.
Category Distribution	Percentage breakdown of social / video / learning / productivity etc.
Peak Browsing Hours	Hour-by-hour visit heatmap; identification of peak and trough activity windows.
Day-wise Patterns	Comparison across weekdays and weekends; identification of most/least active days.
Session Clusters	5 labelled clusters e.g. 'heavy evening social', 'light morning productivity'.
RAM Correlation	Category-level mean and peak browser RAM; top 3 memory-heavy categories/domains.
DL Anomalies	Top-N anomalous sessions with explained reasons (late-night, switching, RAM spike).
Recommendations	≥ 5 personalized, metric-traceable recommendations with priority levels.

10. Output

The pipeline produces the following output files:

Output File	Description
data/browsing_history.csv	Cleaned, privacy-sanitized browsing records with derived columns.
data/sessions.csv	Session-level summary table with all engineered features.
data/clustered_sessions.csv	Sessions enriched with cluster_id, cluster_label, RAM stats, anomaly flags.
reports/cluster_summary.csv	Cluster statistics: count, avg duration, avg visits, dominant category.
reports/ category_ram_stats.csv	Category-wise mean and peak browser RAM usage.
reports/anomaly_sessions.csv	Top-N anomalous sessions with reconstruction error and reason.

reports/final_report.md	Full Markdown report covering all analysis results and recommendations.
dashboard/app.py	Streamlit dashboard — run with: streamlit run dashboard/app.py

11. Advantages

- Fully local and privacy-preserving — no data leaves the user's machine.
- Modular pipeline — each of the 7 modules is independently runnable and testable.
- Combines classical ML (clustering) with deep learning (LSTM / Autoencoder) in one project.
- Real-world data — analysis runs on the learner's own authentic browsing behaviour.
- Configurable — time window, algorithm, threshold and all parameters in a single config.yaml.
- Actionable output — recommendations are directly traceable to measured metrics.
- Interactive dashboard — Streamlit app makes results accessible to non-technical users.
- Educational value — covers the full data science workflow end-to-end.

12. Limitations

- Requires physical access to the browser's SQLite history database file.
- Browsing history records visits, not exact dwell time or scroll depth per page.
- Works only on local desktop data — mobile, tablet, and incognito sessions are excluded.
- RAM log accuracy depends on the sampling interval; fast RAM spikes may be missed.
- Category mapping is heuristic — new or niche domains default to 'other' until mapped.
- Deep learning model quality is constrained by the volume of local browsing data.

13. Future Enhancements

- Real-time monitoring dashboard — live RAM + browsing feed with instant alerts.
- Browser extension integration — capture actual time spent per tab, not just visits.
- Mobile browsing analysis — extend to Android/iOS browser history exports.
- Cross-device aggregation — combine data from multiple browsers and devices.
- Advanced NLP category inference — auto-classify unknown domains using page titles.
- Transformer-based sequence model — replace LSTM with a self-attention architecture for better long-range pattern capture.
- Federated learning — allow privacy-preserving multi-user behaviour comparison.
- Integration with productivity apps — sync with calendar to correlate browsing with meetings/deadlines.
- Automated scheduling — push focus-block recommendations to Google Calendar.

14. Conclusion

The Time-Based Browsing Pattern Analyzer successfully demonstrates how real-world personal data can be transformed into meaningful behavioral intelligence. By integrating data engineering, unsupervised machine learning, and deep learning within a clean, modular pipeline, the system delivers insights that are both technically rigorous and practically useful.

The project covers the complete data science lifecycle — from raw SQLite data extraction through to an interactive Streamlit dashboard — making it a strong showcase of full-stack data science skills. The combination of session clustering, anomaly detection, RAM correlation, and personalized recommendations positions this project at the intersection of Behavior Analytics, Digital Wellbeing, and System Performance Engineering.

With clear pathways for future enhancement — including real-time monitoring, mobile support, and transformer-based models — the system is designed to grow alongside the learner's skills and the evolving demands of the field.

15. References

7. Python Software Foundation — https://docs.python.org
8. Scikit-learn Documentation — https://scikit-learn.org/stable/
9. TensorFlow / Keras Documentation — https://www.tensorflow.org/api_docs
10. Pandas Documentation — https://pandas.pydata.org/docs/
11. psutil Documentation — https://psutil.readthedocs.io/
12. Streamlit Documentation — https://docs.streamlit.io/
13. SQLite Documentation — https://www.sqlite.org/docs.html
14. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly.
15. Research: 'Modeling and Predicting Web Browsing Behavior' — WWW Conference Proceedings.
16. Research: 'Anomaly Detection in User Behavior Using Autoencoders' — IEEE Access.

16. Author

Name	Sathishkumar CB
Specialization	Data Science / Machine Learning
Project	DS105 Final Project — Time-Based Browsing Analyzer
Domain	Behavior Analytics · Digital Wellbeing · Productivity Analytics
Tech Stack	Python · Pandas · NumPy · SQLite · Scikit-learn · TensorFlow · psutil · Streamlit

"Understand how you browse. Optimize how you perform."